



Universidad del Valle de Guatemala
Facultad de Ingeniería
CC3092 — Deep Learning y Sistemas Inteligentes

Laboratorio #7

NLP end-to-end y Embeddings

Nicolás Concuá
Guatemala, octubre de 2026

1. Datos y pipeline de preprocesamiento

Se usó WikiText-103 (raw) para entrenar los embeddings, AG News para la clasificación y GloVe-100 (Wikipedia + Gigaword, 6 B tokens) como referencia. El split de entrenamiento tiene **29,444 artículos, 1.80 M líneas (860 k párrafos) y 101.4 M tokens crudos**, que quedan en **85.8 M** tras la normalización (554 k tipos). Por cómputo se tomaron artículos completos en orden aleatorio hasta **25.0 M tokens** (mínimo pedido: 20 M); los subconjuntos del 25 % y 50 % son prefijos del mismo orden.

- **Normalización (consistente con GloVe 6B).** Se reconstruyen los marcadores de WikiText (@-@ → -, @,@ → ,), todo a minúsculas (GloVe 6B es uncased), contracciones estilo Penn Treebank como el tokenizador de Stanford usado por GloVe (didn't → did n't, king's → king 's), números como dígitos (1990, 3.5), guiones separados y puntuación eliminada (~15 % de los tokens, no aporta a analogías ni similitud). Minúsculas, contracciones y números son lo que debe coincidir con GloVe para comparar sobre las mismas palabras: el 99.7 % de los tokens de nuestro vocabulario existe en GloVe.
- **Tokenizador propio (regex) vs. NLTK (12 oraciones).** Coinciden en contracciones; difieren en guiones (NLTK conserva state-of-the-art), abreviaturas (NLTK reconoce U.S. y p.m.; el propio produce u, s) y puntuación/símbolos, que NLTK emite como tokens.
- **Ley de Zipf.** En log-log la frecuencia vs. rango es casi recta con pendiente $s = 1.07$ entre los rangos 10 y 10 k (Zipf: $s \approx 1$) y más empinada en la cola ($s = 1.75$). Las 10, 1,000 y 30,000 palabras más frecuentes cubren el **24.3 %, 64.6 % y 95.4 %** de los tokens.
- **Vocabulario.** min_count = 1 / 3 / 5 / 10 / 50 da 281 k / 123 k / 90 k / 61 k / 24 k palabras, con 0.0 / 0.8 / 1.2 / 2.0 / 5.2 % de tokens desconocidos en el subconjunto (1.0 / 1.6 / 1.9 / 2.5 / 5.4 % en la validación de WikiText). Se eligió **min_count = 5 (V = 90,273)**: reduce el vocabulario 3× perdiendo 1.2 % de los tokens, y una palabra con < 5 apariciones no tiene contextos para aprender un vector útil.
- **Submuestreo.** Con $P_{\text{keep}}(w) = (\sqrt{f/t} + 1) \cdot t/f$ y $t = 10^{-4}$ se descarta el 96.2 % de "the", 94.1 % de "of" y 93.8 % de "and" (60 % de los tokens se conservan). Ayuda porque las stopwords aparecen en casi todas las ventanas sin aportar información; quitarlas acelera el epoch y amplía el alcance efectivo de la ventana. Con ventana $W = 5$ el corpus produce 239.6 M pares por epoch (ventana fija) o 144.8 M (ventana dinámica de Word2Vec); tras el submuestreo, 141.0 M y **85.6 M**.

2. Investigación: embeddings en PyTorch y gensim

- **nn.Embedding(num_embeddings, embedding_dim, padding_idx, sparse).** Tabla $V \times D$ que devuelve la fila de cada ID (one-hot $\times$ matriz sin construir el one-hot); padding_idx es una fila en ceros sin gradiente y sparse=True da gradientes solo de las filas usadas. **from_pretrained(weights, freeze)** crea la tabla desde vectores entrenados: freeze=True los fija, freeze=False permite fine-tuning [1].
- **nn.EmbeddingBag(mode=mean|sum|max).** Hace lookup + agregación en una sola operación sin crear el tensor (batch, largo, D). Con offsets se pasan todos los IDs del batch en un tensor 1D y la posición donde inicia cada documento, sin padding [1].
- **CBOW vs. skip-gram.** CBOW predice la central con el promedio de sus contextos (rápido); skip-gram predice cada contexto desde la central (mejor en palabras raras) [2]. Negative sampling cambia el softmax sobre V por K clasificaciones binarias contra negativos $\sim \text{count}^{0.75}$ [3].
- **Dos matrices W y W' .** W representa la palabra como centro y W' como contexto; con una sola matriz cada palabra tendría que parecerse a sí misma ($\|w\|^2$ alto) aunque rara vez es su propio contexto. Se conserva W [2][4].
- **Word2Vec vs. GloVe.** Word2Vec es predictivo (SGD ventana a ventana); GloVe es de conteos: construye la matriz global de co-ocurrencias X y ajusta $w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$ por mínimos cuadrados ponderados [5]. Levy & Goldberg mostraron que SGNS factoriza implícitamente la matriz PMI - log K , así que ambos son muy cercanos [4].

3. Skip-gram con negative sampling desde cero e iteraciones

El SGNS usa dos nn.Embedding (entrada W uniforme en $\pm 0.5/D$, salida W' en ceros) y la pérdida $-\log \sigma(u_c^T v_w) - \sum \log \sigma(-u_n^T v_w)$. Cada epoch re-muestra el submuestreo y la ventana dinámica, baraja los pares y entrena por lotes en GPU (MPS) con SGD sobre la pérdida sumada y learning rate lineal $lr \rightarrow 10^{-4}$, como gensim. Cada iteración cambia un hiperparámetro respecto a la base; se evalúa al final de cada epoch con analogías (top 30 k) y WordSim-353 (Tabla 1).

Tabla 1. Iteraciones del SGNS (5 epochs; métricas del último epoch). Base: $d=100$, $W=5$, $K=5$, $t=1e-4$, $lr=0.025$, lote 16,384.

Iteración	Tokens	Pérdida	Sem %	Sin %	Total %	WordSim p	s/epoch	Total (min)	Mem. (GB)
base ($d=100$)	25.0 M	2.197	28.6	45.7	40.2	0.655	50	4.2	5.0
$d=50$	25.0 M	2.233	22.4	36.1	31.7	0.605	26	2.2	5.0
$d=300$	25.0 M	2.147	30.6	48.8	43.0	0.654	79	6.6	5.0
corpus 25 %	6.3 M	2.235	9.8	17.1	14.8	0.522	10	0.9	5.0
corpus 50 %	12.5 M	2.210	19.0	33.4	28.7	0.636	22	1.9	5.0
ventana=2	25.0 M	2.094	23.5	43.9	37.4	0.615	24	2.0	5.0
ventana=10	25.0 M	2.247	36.2	44.3	41.7	0.649	87	7.2	6.0
$K=15$ (lote 16 k)	25.0 M	NaN	0.0	0.0	0.0	—	89	7.4	6.0
$K=15$ (lote 4 k)	25.0 M	3.114	35.9	46.8	43.3	0.654	116	9.7	5.0
$t=1e-3$	25.0 M	2.238	27.6	43.4	38.4	0.647	65	5.4	6.0
$lr=0.05$	25.0 M	2.181	35.6	45.5	42.4	0.663	50	4.2	6.0
gensim (cfg. $K=15$)	25.0 M	—	35.9	46.3	43.0	0.648	17	1.5	CPU

- **Divergencia con $K=15$.** Con lotes de 16 k la pérdida fue inf/NaN desde el inicio: los gradientes del lote se suman y "the" aparece ~3,900 veces como negativo en un mismo lote ($\approx 1,300$ con $K=5$), así que su vector recibe un paso enorme. Word2Vec actualiza par por par y no sufre esto. Con lotes de 4 k fue estable y fue el **mejor modelo (43.3 %)**.
- **¿Pérdida más baja = mejores embeddings? No.** ventana=2 tiene la menor pérdida (2.09) y es de las peores en analogías (predecir vecinos inmediatos es más fácil); $K=15$ tiene la mayor (3.11, suma 15 términos negativos) y es la mejor. Dentro de una corrida, WordSim se estanca desde el epoch 3 mientras la pérdida sigue bajando (curvas en el notebook). Vecinos del mejor modelo: france $\rightarrow$ spain, italy; good $\rightarrow$ bad, decent (los antónimos comparten contexto).

4. Verificación de la aritmética vectorial

La función analogía(a, b, c, k) (3CosAdd sobre vectores normalizados, excluyendo a, b, c) coincide con most_similar de gensim (mismo top-5, diferencia $< 10^{-7}$). En 18 analogías propias de 6 tipos (género, país-capital, gentilicio, comparativo/superlativo, tiempo verbal, plural) el top-1 acertó **13/18 con SGNS y gensim y 15/18 con GloVe**. man:king :: woman:? deja queen 4.^a en el SGNS (1.^a en GloVe) y france:paris :: italy:? da prague (rome en el rango 74); gentilicios y verbos aciertan en los tres. **Sin excluir la consulta**, en 13 de 18 casos el primer lugar es b o c : king – man + woman tiene coseno 0.80 con king y 0.67 con queen.

Tabla 2. Accuracy (%) por categoría en questions-words sobre el vocabulario compartido (30 k palabras; 12,049 de 19,544 preguntas, 61.7 %) y paralelismo (coseno medio entre vectores diferencia).

Categoría	n	SGNS Add	SGNS Mul	gensim Add	GloVe Add	GloVe Mul	Paral. SGNS	Paral. GloVe
capital-common-countries	462	55.2	53.2	52.8	93.7	92.0	0.31	0.59
capital-world	1209	35.2	32.8	35.6	89.2	85.9	0.22	0.57
currency	88	3.4	3.4	2.3	4.5	3.4	0.21	0.26
city-in-state	1745	27.6	26.6	29.1	30.7	26.9	0.19	0.34
family	342	62.9	59.4	57.3	88.3	87.4	0.27	0.51
gram1-adjective-to-adverb	756	15.7	14.3	12.4	29.9	29.6	0.13	0.28
gram2-opposite	306	14.1	11.8	11.1	23.9	20.9	0.08	0.22
gram3-comparative	1190	58.4	56.1	60.8	81.7	79.4	0.32	0.42
gram4-superlative	702	27.5	28.5	28.6	61.3	61.7	0.26	0.47
gram5-present-participle	870	38.6	35.9	41.0	72.1	71.6	0.15	0.32
gram6-nationality-adjective	1299	77.8	76.1	74.7	97.3	95.8	0.45	0.75
gram7-past-tense	1332	60.4	58.1	59.1	58.8	56.3	0.20	0.29

Categoría	n	SGNS Add	SGNS Mul	gensim Add	GloVe Add	GloVe Mul	Paral. SGNS	Paral. GloVe
gram8-plural	992	39.7	34.1	38.4	79.3	75.6	0.14	0.32
gram9-plural-verbs	756	32.1	31.9	33.5	57.7	59.8	0.23	0.36
Semánticas / Sintácticas	3846 / 8203	35.9 / 46.8	34.1 / 44.7	35.9 / 46.3	61.2 / 68.2	58.1 / 66.9	—	—
Total (aleatorio: paral. $\approx$ 0.00)	12049	43.3	41.3	43.0	66.0	64.1	0.23	0.41

- **Lectura.** Las sintácticas superan a las semánticas en los tres modelos: la morfología se repite en cualquier texto, mientras que país–capital es conocimiento del mundo que requiere mucho corpus (la mayor brecha con GloVe: capital-world 35 % vs 89 %). currency falla incluso en GloVe (monedas casi nunca junto a su país, nombres ambiguos) y gram2-opposite es bajo (antónimos en contextos idénticos). gram7 (pasado) es mejor en el SGNS que en GloVe. 3CosMul quedó ~2 puntos debajo de 3CosAdd en este escenario. El paralelismo predice el accuracy: la categoría más paralela (gentilicios) es la mejor y la menos paralela (opposite) de las peores.
- **t-SNE (494 palabras, 10 grupos).** Números (con ordinales en un subgrupo), meses y días, animales, colores, comida, deportes y profesiones forman grupos claros; países y capitales se mezclan por región geográfica (moscow–russia–warsaw, cairo–egypt–damascus), y los verbos forman una región amplia con subgrupos por forma.

5. Pipeline end-to-end: clasificación de AG News

Mismo tokenizador (más decodificación de entidades HTML como #39;); 108,000 / 12,000 (validación estratificada) / 7,600 test. Tokens de AG News fuera del vocabulario: **SGNS y gensim 2.7 %, GloVe 0.57 %** (OOV: basura HTML y entidades de 2004 como peoplesoft o fallujah, ausentes de Wikipedia). Clasificador: EmbeddingBag(mean) → Linear(100, 256) → ReLU → Dropout(0.3) → Linear(256, 4), Adam 1e-3, early stopping por F1 de validación. Baseline: TF-IDF (uni+bigramas) + regresión logística.

Tabla 3. Clasificación de AG News (métricas macro). Test solo para la mejor variante de cada inicialización.

Modelo	Acc val	Prec val	Rec val	F1 val	Params / entrenables	Tiempo (s)	F1 test
TF-IDF + LogReg	92.56	92.54	92.56	92.54	1.68 M / 1.68 M	14	92.21
Aleatorios (entrenables)	91.49	91.49	91.49	91.49	4.28 M / 4.28 M	12	90.74
SGNS congelado	88.58	88.65	88.58	88.57	4.00 M / 26.9 k	12	—
SGNS fine-tuning	91.97	91.98	91.97	91.97	4.00 M / 4.00 M	7	91.48
gensim congelado	88.21	88.27	88.21	88.19	4.00 M / 26.9 k	11	—
gensim fine-tuning	92.04	92.04	92.04	92.04	4.00 M / 4.00 M	7	91.54
GloVe congelado	90.69	90.74	90.69	90.68	5.64 M / 26.9 k	12	—
GloVe fine-tuning	92.49	92.56	92.49	92.48	5.64 M / 5.64 M	11	91.98

En las matrices de confusión Sports es la clase más fácil (~98 % de recall) y el error principal es Business ↔ Sci/Tech (~150 por dirección). SimLex-999: SGNS 0.302, gensim 0.306, GloVe 0.298.

6. Comparación de los tres conjuntos de embeddings

Tabla 4. Comparación entre SGNS propio, Word2Vec de gensim y GloVe (analogías sobre el vocabulario compartido de 30 k).

Métrica	SGNS (propio)	Word2Vec gensim	GloVe-100
Tokens / vocabulario / dimensión	25 M / 90,273 / 100	25 M / 90,273 / 100	6,000 M / 400,000 / 100
Tiempo de entrenamiento	9.7 min (5 epochs)	1.5 min (5 epochs)	~85 min (matriz X) + 50 iter. (≤ 14 min c/u)
Hardware	Apple M4 Pro (GPU, MPS)	Apple M4 Pro (CPU, 12 hilos)	2× Xeon E5-2658, 32 núcleos (autores)
3CosAdd sem / sin / total (%)	35.9 / 46.8 / 43.3	35.9 / 46.3 / 43.0	61.2 / 68.2 / 66.0
3CosMul sem / sin / total (%)	34.1 / 44.7 / 41.3	34.1 / 44.2 / 41.0	58.1 / 66.9 / 64.1
WordSim-353 / SimLex-999 (p)	0.654 / 0.302	0.648 / 0.306	0.533 / 0.298
Paralelismo (coseno medio)	0.226	0.231	0.406
% tokens OOV en AG News	2.70	2.70	0.57
F1 test AG News (fine-tuning)	91.48	91.54	91.98

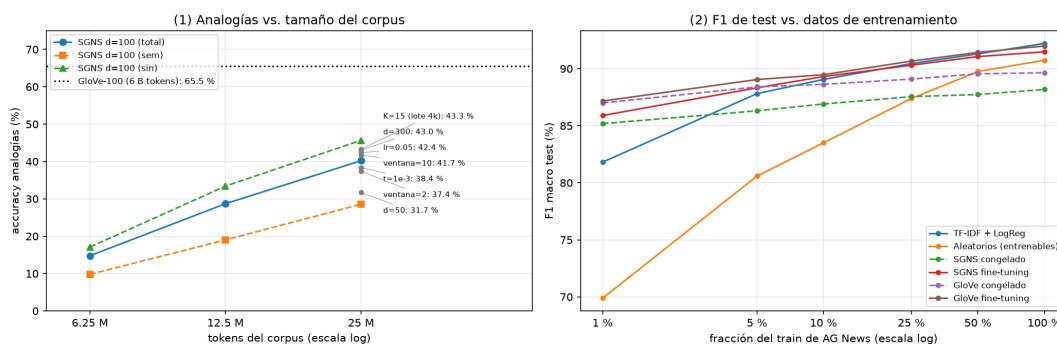


Figura 2. (1) Accuracy de analogías vs. tokens del corpus (GloVe como referencia). (2) F1 de test vs. fracción de datos de AG News.

7. Discusión y conclusiones

- **¿Se cumple la aritmética vectorial?** Aproximadamente: 43 % de las analogías y 13/18 propias, mejor en relaciones regulares (gentilicios 78 %, pasado 60 %, familia 63 %) y peor en monedas y antónimos; las sintácticas (46.8 %) superan a las semánticas (35.9 %).
- **¿El $\approx$ es una igualdad?** No. Sin excluir la consulta, el más cercano a $b - a + c$ suele ser b o c (king 0.80 vs queen 0.67), y los vectores diferencia tienen coseno medio 0.23 (GloVe 0.41), lejos de 1: direcciones compartidas pero ruidosas.
- **Distancia a GloVe (23 puntos).** Se explica sobre todo por el corpus: 6.3 $\rightarrow$ 12.5 $\rightarrow$ 25 M tokens dio 14.8 $\rightarrow$ 28.7 $\rightarrow$ 40.2 % (~13 puntos por duplicación, sin saturar) y GloVe usó 240× más texto, incluyendo noticias. Los hiperparámetros movieron solo 3–6 puntos, y el método no es inferior: el SGNS supera a GloVe en WordSim (0.654 vs 0.533) y en gram7, y lo iguala en SimLex.
- **SGNS vs. gensim.** Equivalentes (43.3 vs 43.0 %, p 0.654 vs 0.648, mismo F1, curvas superpuestas); gensim es ~7× más rápido (C, 12 hilos, par por par) y el SGNS por lotes puede divergir con K y lotes grandes.
- **Dimensión, ventana y negativos.** d 50 $\rightarrow$ 300: 31.7 $\rightarrow$ 43.0 % con rendimientos decrecientes. Ventana 2 $\rightarrow$ 10: semánticas 23.5 $\rightarrow$ 36.2 % y sintácticas ~44 %: las ventanas pequeñas favorecen lo sintáctico y las grandes lo semántico. $K=15$: 43.3 % (sobre todo semánticas).
- **AG News.** Con todos los datos TF-IDF fue el mejor (92.2 %): la tarea depende de palabras clave y 108 k ejemplos bastan incluso para una tabla aleatoria (90.7 %); los preentrenados superaron a los aleatorios y fine-tuning convino siempre (SGNS 88.2 $\rightarrow$ 91.5). **Con 1 % de los datos** los preentrenados ganan con claridad (GloVe 87.2 %, SGNS 85.9 % vs TF-IDF 81.8 % y aleatorios 69.9 %) y congelar es casi igual que hacer fine-tuning (85.2 vs 85.9) con menos riesgo de sobreajuste.

Referencias

- [1] PyTorch Documentation. torch.nn.Embedding y torch.nn.EmbeddingBag. <https://pytorch.org/docs/stable/nn.html>
- [2] Mikolov, T., Chen, K., Corrado, G. y Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781.
- [3] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. y Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. NeurIPS.
- [4] Levy, O. y Goldberg, Y. (2014). Neural Word Embedding as Implicit Matrix Factorization. NeurIPS.
- [5] Pennington, J., Socher, R. y Manning, C. (2014). GloVe: Global Vectors for Word Representation. EMNLP.
- [6] Řehůřek, R. y Sojka, P. gensim: Word2Vec y KeyedVectors. <https://radimrehurek.com/gensim/>

Enlace al repositorio: <https://github.com/nicoCT04/Lab7-DL>